

IK2218/EP2120 Protocols and Principles of the Internet

Homework 3

Solutions due: 19:00, September 28, 2026

Review due: 19:00, September 30, 2026

1. HTTP Cookies (25 p)

You visit the web shop `www.duckshop.example` for the first time, and your browser receives the following response:

```
HTTP/1.1 200 OK
Date: Tue, 22 Sep 2026 09:15:00 GMT
Set-Cookie: SID=31d4d96e; Max-Age=2592000
Content-Type: text/html
```

...

From then on, every request your browser sends to `www.duckshop.example` contains this header line:

```
Cookie: SID=31d4d96e
```

The page also contains an image hosted on a different server, `tracker.example`. Your browser fetches that image with the following request:

```
GET /pixel.gif HTTP/1.1
Host: tracker.example
Referer: http://www.duckshop.example/swordfish.html
```

This is the response:

```
HTTP/1.1 200 OK
Set-Cookie: uid=99f3ac21; Max-Age=31536000
Content-Type: image/gif
```

...

Note: The HTTP headers above have elements that are not covered in any detail in the course material. You are encouraged to learn more about those elements on your own.

- (a) HTTP is said to be a stateless protocol. Explain what that means, and explain how the web shop can still keep track of what you have put in your shopping cart, for example. (5 p)
- (b) Where is the “state” stored in this example? Identify the places, and explain what is stored in each of them. (5 p)
- (c) A week later you visit a completely different web site, `www.horses.example`, which also has an image from `tracker.example` on its pages. Explain what `tracker.example` can learn about you, and how. Note that you have never visited `tracker.example` yourself. (10 p)
- (d) In the EU, storing a cookie on a user’s computer requires that the user is informed and consents to it, and a user has the option to reject cookies. What would happen if the user rejects any the two cookies described above? Motivate your answer. (5 p)

2. Keeping State Without Cookies (25 p)

You are building a web application in which the server needs to recognize a returning user. You dislike cookies, so you decide to investigate other possible ways of identifying users.

- (a) First you try with a scheme based on IP addresses: Users are identified by the source IP addresses of their HTTP requests. Explain why IP addresses is not a good way of identifying users. Describe two different scenarios in which two HTTP users cannot be distinguished by the IP source addresses in their HTTP requests. Also describe one scenario where the same user may send requests with different IP addresses. (5 p)

- (b) You redesign your scheme and decide instead to use TCP connections to distinguish users. In this new scheme, all HTTP requests that arrive on the same TCP connection are considered as belonging to the same user, and requests that arrive on different TCP connections are from different users. (5 p)
1. You realize that this may not work with non-persistent connection, so you configure you server to use persistent connections. Explain what the problem is with non-persistent connections in this scheme.
 2. To your disappointment, you find that it does not work well with persistent connections either. Explain.
- (c) After contemplating your initial failures, you come up with a brilliant idea: You put a user identifier in each URL generated on your server's pages. It works this way: assume that a web page on the server consists of several URL links, for example one link for each item in a web shop. The web page is computed dynamically when a user requests it, so it will be unique for each user, and each item URL on the page will contain the user's identifier. So each user would get its own version of the web page, where each link to an item in the shop contains the user's identifier, for example as a query parameter. (8 p)
- This design has actually been tried before (although you were not aware of it), and it does work, but it has significant limitations. Give two distinct problems with it.
- (d) If HTTP had been designed as a "stateful" protocol, it might have saved you all this trouble. Then HTTP could keep track of user sessions for you. However, there are several reasons why HTTP was not designed this way. Instead, it is "stateless". Give two advantages of a stateless design, and one thing that it makes harder. (7 p)

3. Server Concurrency (25 p)

A TCP server handles one client at a time: it accepts a connection, reads the request, processes it, sends the response, closes the connection, and then accepts the next connection. Processing a request takes 5 ms, and everything else takes a negligible amount of time.

- (a) An advantage with this design is that it is simple – one single thread of processing, which handles requests sequentially. (5 p)
- But what is the highest rate of requests per second this server can handle? Explain how long a client would wait for a response in the cases when requests arrive at the server at the rate of 100 per second, at 190 per second, and at 250 per second. Keep in mind that clients send their requests independently of each other, so requests will arrive at the server in a random, unpredictable manner. In particular, they do not arrive at a regular pace.
- (b) The server is rewritten to create a new process for each connection, so that several requests are processed at the same time. This would remove the limit above, but may introduce new problems. (5 p)
1. Explain what now limits the number of clients the server can handle at the same time.
 2. What is the problem with this design for a server with a large number of connections open at once, most of which are idle?
- (c) A third design uses a single process that keeps track of all the connections at once and handles data on any of them as soon as it can. It works like this: The server monitors all connections. Whenever it detects that data has arrived on a connection, the process deals with that connection. Then, the process finds the next connection where data has arrived, and deals with that connection, and so on. (8 p)
1. Explain why this can serve more clients than the design above (with one process per connection).
 2. The system function to read data on a socket is a *blocking* operation. Explain why this is a problem here, and why some other mechanism than blocking read operations is required for this scheme to work.

- (d) The problem of handling many clients is mainly an issue with TCP. A TCP server needs a separate socket for every connected client, while a UDP server can serve all clients with a single socket. Explain where the difference comes from. Why does TCP require one socket per client, while UDP can use the same socket for all clients? (7 p)

4. Remote Login and Port Forwarding (25 p)

The IMAP server `imap.feathers.org` only accepts connections from inside the organization's network, on the standard IMAP port 143. Alice is at home, and wants to read her mail that are stored on the IMAP server. She has an account on the machine `shell.feathers.org`, which is inside the organization's network and reachable from the outside over SSH.

Alice therefore runs the following command on her laptop:

```
ssh -L 1430:imap.feathers.org:143 alice@shell.feathers.org
```

This sets up an SSH connection with port forwarding: when a connection is made to port 1430 on Alice's laptop, the connection is forwarded over SSH to port 143 on `imap.feathers.org`. Alice configures her mail program to use `localhost` as her IMAP server, on port 1430, and then she can access `imap.feathers.org`.

- (a) There are three TCP connections involved when Alice's mail program fetches mail from `imap.feathers.org`. List the connections, and give the endpoints of each one: which machines, and which port numbers. (If a port number cannot be determined by the description above, say so, and explain what kind of port number it is.) (10 p)
- (b) For each of the three connections, state which party is the client and which is the server, according to the Internet client/server model. This is not the same thing as who is the SSH client and who is the SSH server. Explain how the SSH client and SSH server in this scenario relate to the Internet client/server model. (5 p)
- (c) Which parts of the path are encrypted, and which are not? What would someone who is able to eavesdrop on the organization's internal network be able to see? (5 p)
- (d) Telnet is an older protocol for remote login. It is considered insecure and has largely been replaced by SSH. What are the two properties that SSH adds that make the communication secure? Explain why both are needed. (5 p)